COMPLETE TECHNICAL DOCUMENTATION

IoT Gateway Stack

OpenThread Border Router with MQTT and Node-RED Dashboard

QPG6200L IoT Mesh Network Project

Document Version	1.0 (Comprehensive Edition)
Date Created	March 9, 2026
Classification	Internal Technical Documentation
Target Audience	Engineers with basic Linux knowledge
Estimated Setup Time	2-3 hours

This document provides step-by-step instructions to replicate the complete IoT Gateway Stack setup. Every command, configuration file, and troubleshooting step is documented with detailed explanations of what each component does and why it is necessary.

Table of Contents

1. Executive Summary and System Overview
2. Understanding the Technology Stack
3. Hardware Requirements and Specifications
4. Prerequisites and Development Environment Setup
5. Phase 1: Building and Flashing the nRF52840 RCP Firmware
6. Phase 2: Raspberry Pi Operating System Installation
7. Phase 3: OpenThread Border Router Installation and Configuration
8. Phase 4: Mosquitto MQTT Broker Installation and Configuration
9. Phase 5: Node-RED and Dashboard 2.0 Installation
10. System Integration and End-to-End Testing
11. Issues Encountered and Detailed Resolutions
12. Final System Configuration Summary
13. Quick Start Guide: Daily Operations
14. Troubleshooting Guide
15. Appendix A: Complete Command Reference
16. Appendix B: Configuration File Contents
17. Appendix C: Glossary of Terms

1. Executive Summary and System Overview

1.1 Purpose of This Document

This document serves as a complete, reproducible guide for setting up an IoT Gateway Stack. It was created during an actual deployment on March 9, 2026, and includes every command executed, every error encountered, and every solution applied. A person with basic Linux command-line knowledge should be able to follow this guide and achieve an identical working system.

1.2 What This System Does

The IoT Gateway Stack enables wireless IoT devices (built with Qorvo QPG6200L chips) to communicate over a Thread mesh network and send their data to a visual dashboard. Here is what each part does:

- **Thread Mesh Network:** A low-power wireless network protocol (similar to Wi-Fi but designed for IoT devices). Devices can talk to each other and relay messages, creating a self-healing mesh. If one device fails, messages route around it.
- **Border Router:** The bridge between the Thread network (wireless) and your regular Ethernet/Wi-Fi network. Without this, your Thread devices cannot reach the internet or your local network.
- **MQTT Broker:** A message routing service. Devices publish data to topics (like home/motion/sensor1), and other applications subscribe to those topics to receive the data. Think of it like a post office for IoT messages.
- **Node-RED Dashboard:** A visual programming tool that receives MQTT messages and displays them on a web-based dashboard with gauges, charts, and buttons. This is what you see on your tablet or phone.

1.3 System Architecture Diagram

Data flows through the system in this order:

```
┌──────────────────────────────────────────────────────────────┐
│  QPG6200L IoT Devices (Motion Sensor, Doorbell, etc.)        │
│  - Run on coin cell batteries for months/years               │
│  - Communicate using Thread protocol (IEEE 802.15.4)         │
└──────────────────────────────────────────────────────────────┘
                                    │
                              Wireless (2.4 GHz)
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────┐
│  nRF52840 USB Dongle (Radio Co-Processor / RCP)              │
│  - Receives Thread radio signals                             │
│  - Converts to serial data via USB                           │
│  - Uses Spinel protocol to communicate with Pi               │
└──────────────────────────────────────────────────────────────┘
                                    │
                               USB Serial Cable
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────┐
│  Raspberry Pi 4 (Gateway Computer)                           │
│   ┌──────────────────────────────────────────────────────┐   │
```

```
│  ┌──────────────────────────────────────────────────────┐  │
│  │ OTBR (OpenThread Border Router)                      │  │
│  │ - Manages Thread network                             │  │
│  │ - Routes IPv6 between Thread and Ethernet            │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  ┌──────────────────────────────────────────────────────┐  │
│  │ Mosquitto MQTT Broker (Port 1883)                    │  │
│  │ - Routes publish/subscribe messages                  │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  ┌──────────────────────────────────────────────────────┐  │
│  │ Node-RED + Dashboard 2.0 (Port 1880)                 │  │
│  │ - Visual flow programming                            │  │
│  │ - Web-based dashboard with gauges and charts         │  │
│  └──────────────────────────────────────────────────────┘  │
└────────────────────────────────────────────────────────────┘
```

1.4 Final System Status

After completing all steps in this guide, you will have:

Component	Status	How to Access
nRF52840 Dongle	Flashed & Working	Plugged into Pi USB, detected as /dev/ttyACM0
Thread Network	Active (Leader)	Channel 18, PAN ID 0x1905
OTBR Web GUI	Running	http://192.168.1.69 (browser)
Mosquitto MQTT	Running	Port 1883 (mqtt://192.168.1.69:1883)
Node-RED Editor	Running	http://192.168.1.69:1880 (browser)
Dashboard	Ready	http://192.168.1.69:1880/dashboard (browser)

2. Understanding the Technology Stack

Before diving into the setup, it is important to understand what each technology does and why it was chosen. This section explains each component in detail.

2.1 Thread Protocol

What is Thread?

Thread is a wireless networking protocol designed specifically for IoT devices. It was created by the Thread Group (which includes Google, Apple, Amazon, and others) to address the limitations of existing protocols like Wi-Fi, Zigbee, and Bluetooth.

Why Thread Instead of Wi-Fi?

- **Power Consumption:** Wi-Fi devices need to maintain a constant connection and typically require wall power. Thread devices can run on coin cell batteries for years because they only transmit when needed.
- **Mesh Networking:** Wi-Fi uses a hub-and-spoke model where every device talks directly to the router. Thread creates a mesh where devices relay messages for each other, extending range and improving reliability.
- **Self-Healing:** If a Thread device fails or moves, the network automatically reconfigures itself to route around the problem. Wi-Fi networks do not have this capability.
- **IPv6 Native:** Thread uses standard IPv6 addressing, making it easy to integrate with existing IT infrastructure. Each device gets its own IPv6 address.

Thread Network Concepts

- **Leader:** One device in the Thread network is elected as the Leader. It manages network-wide configuration and assigns addresses. In our setup, the Border Router acts as the Leader.
- **Router:** Devices that can relay messages for other devices. They are mains-powered because relaying requires more energy.
- **End Device:** Battery-powered devices that send and receive their own messages but do not relay for others. Our sensors will be End Devices.
- **Border Router:** A special device that bridges the Thread network to other IP networks (like your home Ethernet/Wi-Fi). Without a Border Router, Thread devices cannot communicate with the internet or your local network.
- **Network Key:** A 128-bit encryption key that all devices must have to join the network. This ensures security - unauthorized devices cannot join or eavesdrop.

2.2 Radio Co-Processor (RCP) Architecture

The Raspberry Pi does not have a built-in IEEE 802.15.4 radio (the radio type used by Thread). We need to add this capability using an external USB dongle. There are two ways to do this:

- **Network Co-Processor (NCP):** The dongle runs the full Thread stack. The Pi just sends high-level commands. This is simpler but less flexible.
- **Radio Co-Processor (RCP):** The dongle only handles the low-level radio operations (transmit/receive packets). The Pi runs the full Thread stack. This gives us more control and allows for easier updates. THIS IS WHAT WE USE.

The nRF52840 dongle communicates with the Raspberry Pi using the Spinel protocol over a USB serial connection. Spinel is a simple binary protocol designed for this purpose.

2.3 OpenThread Border Router (OTBR)

OpenThread is Google's open-source implementation of the Thread protocol. OTBR (OpenThread Border Router) is a complete software package that turns a Raspberry Pi (with an RCP dongle) into a Thread Border Router.

OTBR Components

- **otbr-agent:** The main service that runs the Thread stack, manages the RCP dongle, handles IPv6 routing, and creates the wpan0 network interface. This is the core of OTBR.
- **otbr-web:** A web server that provides a graphical interface for managing the Thread network. You can form networks, view connected devices, and configure settings through a browser.

2.4 MQTT (Message Queuing Telemetry Transport)

MQTT is a lightweight messaging protocol designed for IoT applications. It uses a publish/subscribe model rather than the traditional client/server model.

How MQTT Works

1. **Topics:** Messages are organized into topics, which are like channels. Example: home/living-room/motion or home/front-door/doorbell
2. **Publish:** A device sends (publishes) a message to a topic. Example: The motion sensor publishes {"motion": true, "distance": 42} to home/living-room/motion
3. **Subscribe:** Applications subscribe to topics they care about. The broker sends them all messages published to those topics.
4. **Broker:** The MQTT broker (Mosquitto in our case) sits in the middle, receiving all published messages and forwarding them to all subscribers of each topic.

Why Mosquitto?

Mosquitto is an open-source MQTT broker from the Eclipse Foundation. It is lightweight, reliable, widely used, and runs well on Raspberry Pi. It is the industry standard for small to medium IoT deployments.

2.5 Node-RED

Node-RED is a visual programming tool originally developed by IBM. It allows you to wire together IoT devices, APIs, and services using a browser-based flow editor. Instead of writing code, you drag and drop nodes and connect them with wires.

Why Node-RED?

- **Visual Programming:** Non-programmers can create IoT applications by connecting nodes visually.
- **MQTT Integration:** Built-in MQTT nodes make it trivial to subscribe to topics and process messages.

- **Dashboard 2.0:** The FlowFuse Dashboard extension adds drag-and-drop UI widgets (gauges, charts, buttons) that automatically update in real-time.
- **Extensible:** Thousands of community-contributed nodes for connecting to databases, cloud services, and more.

3. Hardware Requirements and Specifications

3.1 Gateway Hardware (Required)

These are the physical components you need for the gateway:

Component	Specification	Purpose and Notes
Raspberry Pi 4	Model B, 2GB+ RAM recommended	The main gateway computer. Runs Linux and all our software. 2GB RAM is sufficient; 4GB or 8GB provides more headroom.
MicroSD Card	16GB minimum, 32GB recommended, Class 10 or faster	Stores the operating system and all software. Faster cards improve boot time and overall performance.
nRF52840 USB Dongle	Nordic Semiconductor PCA10059	Provides the IEEE 802.15.4 radio for Thread. Must be flashed with RCP firmware before use. Available from Mouser, DigiKey, or direct from Nordic.
USB-C Power Supply	5V 3A (15W) minimum, official Raspberry Pi PSU recommended	Powers the Raspberry Pi. Underpowered supplies cause instability. The official PSU is reliable.
Ethernet Cable	Cat5e or Cat6, length as needed	Connects Pi to your network router/switch. Wired connection is more reliable than Wi-Fi for a gateway.
Display Device	Lenovo Tab M11 or any device with a web browser	For viewing the dashboard. Can be a tablet, phone, or computer. Any modern web browser works.

3.2 Development/Build Computer (Required for Initial Setup)

You need a separate computer to compile the RCP firmware for the nRF52840 dongle. This computer is only needed during initial setup.

Component	Specification	Purpose and Notes
Linux Computer	Ubuntu 20.04 or newer recommended	Used to compile the OpenThread RCP firmware. Can be a dedicated machine, a VM, or Windows WSL2. Our setup used a Dell Latitude 7280 running Ubuntu.
SD Card Reader	USB or built-in, microSD compatible	For writing the Raspberry Pi OS image to the microSD card.

ℹ **NOTE:** The build computer only needs to be used once to flash the nRF52840 dongle. After that, all operations are performed on or via the Raspberry Pi.

4. Prerequisites and Development Environment Setup

Before starting the installation phases, you need to prepare your build computer with the necessary software tools. This section covers everything you need to install.

4.1 Required Software on Build Computer

Open a terminal on your Ubuntu build computer and install the following packages:

Step 1: Update the package list (ensures you get the latest versions):

```
sudo apt update
```

Explanation: This command downloads the latest list of available packages from Ubuntu's servers. Always run this before installing new software.

Step 2: Install build tools:

```
sudo apt install -y build-essential cmake ninja-build
```

Explanation of each package:

- **build-essential:** Installs GCC compiler, make, and other basic build tools needed to compile C/C++ code.
- **cmake:** A cross-platform build system generator. The OpenThread project uses CMake to manage its build process.
- **ninja-build:** A fast build system. CMake generates Ninja build files, which compile code faster than traditional Makefiles.

Step 3: Install Python 3 and pip (package manager for Python):

```
sudo apt install -y python3 python3-pip
```

Explanation: Python is used by several build scripts, and pip is needed to install the nrfutil tool later.

Step 4: Install Git (version control system):

```
sudo apt install -y git
```

Explanation: Git is used to download (clone) the OpenThread source code from GitHub.

Step 5: Install the ARM cross-compiler toolchain:

```
sudo apt install -y gcc-arm-none-eabi binutils-arm-none-eabi
```

Explanation: The nRF52840 uses an ARM Cortex-M4 processor. Since your build computer uses an x86/x64 processor, you need a cross-compiler that runs on x86 but generates ARM code.

- **gcc-arm-none-eabi:** The GNU C compiler for ARM microcontrollers (bare-metal, no operating system).
- **binutils-arm-none-eabi:** Binary utilities (linker, objcopy, etc.) for ARM. We use objcopy to convert the compiled binary to Intel HEX format.

Step 6: Install nrfutil (Nordic Semiconductor's flashing tool):

```
pip3 install --user nrfutil==6.1.7
```

Explanation: nrfutil is Nordic Semiconductor's command-line tool for creating DFU (Device Firmware Update) packages and flashing them to nRF devices via USB bootloader.

- **--user:** Installs to ~/.local/bin instead of system-wide, avoiding permission issues.
- **==6.1.7:** Specifies version 6.1.7, which is compatible with the nRF52840 dongle's USB bootloader.

Step 7: Add ~/.local/bin to your PATH (if not already):

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Explanation: This ensures your shell can find the nrfutil command. The first command adds the export to your .bashrc file (runs every time you open a terminal). The second command reloads .bashrc immediately.

Step 8: Verify nrfutil is installed correctly:

```
nrfutil version
```

Expected output: nrfutil version 6.1.7

5. Phase 1: Building and Flashing the nRF52840 RCP Firmware

This phase compiles the OpenThread RCP firmware from source code and flashes it onto the nRF52840 USB dongle. After this phase, the dongle will be ready to use as a Thread radio.

5.1 Understanding What We Are Doing

The nRF52840 dongle comes from the factory with either no firmware or demo firmware. We need to flash it with OpenThread RCP firmware, which turns it into a Radio Co-Processor that the Raspberry Pi can use for Thread communication.

The process involves: (1) downloading the OpenThread source code, (2) compiling it for the nRF52840 chip, (3) converting the compiled binary to a format the USB bootloader understands, and (4) flashing it via USB.

5.2 Clone the OpenThread nRF528xx Repository

Navigate to your home directory and clone the repository:

```
cd ~
git clone --recursive https://github.com/openthread/ot-nrf528xx.git
```

Explanation:

- **cd ~:** Changes to your home directory (e.g., /home/yourname).
- **git clone:** Downloads a copy of the repository from GitHub.
- **--recursive:** Also downloads submodules (dependencies). The OpenThread project includes the main OpenThread stack as a submodule.

This may take a few minutes depending on your internet speed. The repository is approximately 200-300 MB.

5.3 Navigate to the Repository

```
cd ot-nrf528xx
```

5.4 Build the RCP Firmware

Run the build script with the correct options:

```
./script/build nrf52840 USB_trans -DOT_BOOTLOADER=USB
```

Explanation of each argument:

- **./script/build:** The build script provided by the OpenThread nRF528xx repository.
- **nrf52840:** Specifies the target chip. The nRF52840 is Nordic's most capable Bluetooth/Thread chip.
- **USB_trans:** Specifies USB transport. The dongle communicates with the Raspberry Pi over USB serial.
- **-DOT_BOOTLOADER=USB:** Tells the build system to configure the firmware for USB DFU bootloader. The dongle has a built-in USB bootloader that allows firmware updates without special hardware.

Build Progress: The build will show progress like "[123/924] Building CXX object...". This takes 5-15 minutes depending on your computer's speed.

Expected Result: The build completes with "[924/924] Linking CXX executable bin/ot-rcp". The compiled firmware is located at build/bin/ot-rcp.

⚠ **WARNING:** You may see linker warnings about unimplemented POSIX functions (_close, _fstat, _read, _write). These are expected and harmless for bare-metal firmware.

5.5 Convert to Intel HEX Format

The build produces an ELF binary, but the USB bootloader expects Intel HEX format:

```
arm-none-eabi-objcopy -O ihex build/bin/ot-rcp build/bin/ot-rcp.hex
```

Explanation:

- **arm-none-eabi-objcopy:** A tool that converts between binary formats.
- **-O ihex:** Output format is Intel HEX (a text-based format commonly used for programming microcontrollers).
- **build/bin/ot-rcp:** Input file (ELF format).
- **build/bin/ot-rcp.hex:** Output file (Intel HEX format).

5.6 Create DFU Package

Package the HEX file for USB DFU flashing:

```
nrfutil pkg generate --hw-version 52 --sd-req=0x00 --application
build/bin/ot-rcp.hex --application-version 1 build/bin/ot-rcp.zip
```

Explanation of each argument:

- **pkg generate:** Creates a DFU package (a ZIP file containing the firmware and metadata).
- **--hw-version 52:** Hardware version identifier (52 for nRF52 series).
- **--sd-req=0x00:** SoftDevice requirement. 0x00 means no SoftDevice is required (we are not using Bluetooth in the RCP).
- **--application:** Path to the HEX file to include.
- **--application-version 1:** Version number embedded in the package (can be any number).
- **build/bin/ot-rcp.zip:** Output ZIP file.

5.7 Put the Dongle in Bootloader Mode

Physical steps:

1. Plug the nRF52840 dongle into a USB port on your build computer.
2. Press the small RESET button on the dongle (it is a tiny button near the USB connector).
3. The LED on the dongle should start blinking RED. This indicates bootloader mode.

✓ **TIP:** If the LED does not blink, try holding the RESET button for 1-2 seconds. Some dongles require a longer press.

5.8 Verify the Dongle is Detected

```
ls /dev/ttyACM*
```

Expected output: /dev/ttyACM0 (or similar). This is the USB serial port created by the dongle's bootloader.

5.9 Flash the Firmware

Use nrfutil to flash the DFU package:

```
sudo $(which nrfutil) dfu usb-serial -pkg build/bin/ot-rcp.zip -p
/dev/ttyACM0
```

Explanation:

- **sudo:** Runs with administrator privileges (needed to access USB serial ports).
- **$(which nrfutil):** Finds the full path to nrfutil. This is needed because sudo may not have ~/.local/bin in its PATH.
- **dfu usb-serial:** Perform DFU update over USB serial connection.
- **-pkg:** Path to the DFU package (ZIP file).
- **-p:** Serial port to use (/dev/ttyACM0).

Expected output: Progress bar showing "Upgrading target..." followed by "Device programmed."

> ✓ **TIP:** After successful flashing, the LED on the dongle will STOP blinking. This is correct behavior - the RCP firmware does not include any LED code.

5.10 Verify the Flash Was Successful

Unplug and replug the dongle, then verify it appears as a serial device:

```
ls /dev/ttyACM*
```

You should see /dev/ttyACM0. The dongle is now ready to be moved to the Raspberry Pi.

6. Phase 2: Raspberry Pi Operating System Installation

This phase prepares the Raspberry Pi with its operating system and enables remote access via SSH.

6.1 Download Raspberry Pi Imager

Download Raspberry Pi Imager from: https://www.raspberrypi.com/software/

Install it on your build computer following the instructions for your operating system.

6.2 Write the Operating System

1. Insert your microSD card into the card reader on your build computer.
2. Open Raspberry Pi Imager.
3. Click "CHOOSE OS" → "Raspberry Pi OS (other)" → "Raspberry Pi OS (64-bit)".
4. Click "CHOOSE STORAGE" and select your microSD card.
5. Click the gear icon (⚙) to open advanced options.
6. Enable SSH (Use password authentication).
7. Set username: pi
8. Set password: raspberry (or a password of your choice)
9. Optionally configure Wi-Fi (we use Ethernet, so this is not necessary).
10. Click "SAVE" to close advanced options.
11. Click "WRITE" and wait for the process to complete (5-15 minutes).

⚠ **WARNING:** The advanced options may not always apply correctly. See Section 11 for troubleshooting if SSH does not work.

6.3 (If Needed) Manually Enable SSH

If SSH is not working after boot, you can enable it manually by creating an empty file called 'ssh' on the boot partition:

```
sudo touch /media/$(whoami)/bootfs/ssh
```

Explanation: The Raspberry Pi checks for this file on boot. If it exists, SSH is enabled. The file is automatically deleted after enabling SSH.

6.4 (If Needed) Manually Create User Account

If you get "Permission denied" when logging in, create the user manually by creating a file called 'userconf.txt' on the boot partition:

```
echo 'pi:'$(echo 'raspberry' | openssl passwd -6 -stdin) | sudo tee
/media/$(whoami)/bootfs/userconf.txt
```

Explanation:

- **pi:** The username.
- **openssl passwd -6:** Generates a SHA-512 encrypted password hash.
- **tee:** Writes the output to the file (with sudo for permissions).

6.5 Boot the Raspberry Pi

1. Safely eject the SD card from your build computer.
2. Insert the SD card into the Raspberry Pi.
3. Connect the Ethernet cable to the Pi and to your network switch/router.
4. Plug in the nRF52840 dongle (that you flashed earlier) to any USB port on the Pi.
5. Connect the USB-C power supply to the Pi.
6. Wait 60-90 seconds for the first boot to complete.

6.6 Find the Pi's IP Address

You need to find the IP address assigned to the Raspberry Pi by your router. Options include:

- Check your router's admin interface for connected devices.
- Use a network scanner app on your phone.
- Try ping raspberrypi.local (works if mDNS is available on your network).

In our setup, the Pi was assigned IP address: 192.168.1.69

6.7 Connect via SSH

From your build computer, open a terminal and connect:

```
ssh pi@192.168.1.69
```

Type 'yes' when asked about the fingerprint, then enter the password (raspberry).

You should now have a command prompt on the Raspberry Pi.

6.8 Update the System

```
sudo apt update
sudo apt upgrade -y
```

Explanation: This updates the package list and installs all available updates. This may take 5-10 minutes.

6.9 Verify the Dongle is Detected

```
ls /dev/ttyACM*
```

Expected output: /dev/ttyACM0. This confirms the nRF52840 dongle is recognized.

7. Phase 3: OpenThread Border Router Installation and Configuration

This phase installs and configures the OpenThread Border Router, which manages the Thread network and bridges it to your Ethernet network.

7.1 Clone the OTBR Repository

```
cd ~
git clone --depth=1 https://github.com/openthread/ot-br-posix
cd ot-br-posix
```

Explanation: --depth=1 performs a shallow clone (only the latest commit), which is faster and uses less disk space.

7.2 Run Bootstrap

```
./script/bootstrap
```

What this does: Installs all required dependencies including compilers, libraries, and tools. This takes 10-20 minutes.

7.3 Install Node.js and npm (If Not Already Installed)

The web GUI requires npm. If the setup script fails with 'npm not found', install it:

```
sudo apt install -y nodejs npm
```

7.4 Run Setup

```
INFRA_IF_NAME=eth0 WEB_GUI=1 ./script/setup
```

Explanation:

- **INFRA_IF_NAME=eth0:** Specifies the infrastructure interface (your Ethernet connection). This is where Thread devices will be bridged to.
- **WEB_GUI=1:** Enables the web-based management interface.
- **./script/setup:** Compiles and installs OTBR. This takes 30-60 minutes.

7.5 Reboot

```
sudo reboot
```

Wait 60 seconds, then reconnect via SSH.

7.6 Form the Thread Network

Run these commands to create a new Thread network:

```
sudo ot-ctl dataset init new
```

What it does: Generates random network credentials (channel, PAN ID, network key, etc.)

```
sudo ot-ctl dataset commit active
```

What it does: Saves the credentials as the active dataset.

```
sudo ot-ctl ifconfig up
```

What it does: Enables the Thread network interface (wpan0).

```
sudo ot-ctl thread start
```

What it does: Starts Thread networking. The device will become the Leader since it is the first (and only) device.

7.7 Verify Thread Network Status

```
sudo ot-ctl state
```

Expected output: leader

7.8 Record Network Credentials

Run these commands and save the output - you will need these values for your QPG6200L devices:

```
sudo ot-ctl channel
sudo ot-ctl panid
sudo ot-ctl networkkey
```

Our values (yours will be different): Channel: 18, PAN ID: 0x1905, Network Key: 67b33c6dca3b4ccf96a56c318b887c39

7.9 Fix Web GUI Network Binding (Important!)

By default, the web GUI only listens on localhost (127.0.0.1), so you cannot access it from another computer. Fix this:

```
echo 'OTBR_WEB_OPTS="-I wpan0 -a 0.0.0.0"' | sudo tee /etc/default/otbr-web
sudo systemctl restart otbr-web
```

Explanation:

- **-I wpan0:** Specifies the Thread interface name.
- **-a 0.0.0.0:** Listen on all network interfaces (not just localhost).

You can now access the OTBR web interface at http://192.168.1.69 (replace with your Pi's IP).

8. Phase 4: Mosquitto MQTT Broker Installation and Configuration

8.1 Install Mosquitto

```
sudo apt install -y mosquitto mosquitto-clients
```

What gets installed: mosquitto (the broker daemon) and mosquitto-clients (command-line tools for testing).

8.2 Configure Mosquitto for Network Access

Mosquitto 2.x defaults to localhost-only for security. We need to enable network access:

```
sudo nano /etc/mosquitto/mosquitto.conf
```

Add these lines at the end of the file:

```
listener 1883 0.0.0.0
allow_anonymous true
```

Explanation:

- **listener 1883 0.0.0.0:** Listen on port 1883 on all network interfaces.
- **allow_anonymous true:** Allow connections without username/password. (For production, configure authentication.)

Save and exit (Ctrl+O, Enter, Ctrl+X), then restart Mosquitto:

```
sudo systemctl restart mosquitto
```

8.3 Test Mosquitto

In one terminal, subscribe to a test topic:

```
mosquitto_sub -t test/hello -v
```

In another terminal (SSH session), publish a message:

```
mosquitto_pub -t test/hello -m "Hello from Mosquitto!"
```

The subscriber should display: test/hello Hello from Mosquitto!

9. Phase 5: Node-RED and Dashboard 2.0 Installation

9.1 Install Node-RED

```
bash <(curl -sL https://raw.githubusercontent.com/node-red/linux-installers/master/deb/update-nodejs-and-nodered)
```

When prompted:

- "Are you really sure you want to do this?" → Type y and press Enter
- "Would you like to install the Pi-specific nodes?" → Type y and press Enter

Installation takes 10-20 minutes.

9.2 Enable and Start Node-RED

```
sudo systemctl enable nodered.service
sudo systemctl start nodered.service
```

What this does: enable makes Node-RED start automatically on boot. start starts it immediately.

9.3 Access Node-RED

Open a web browser and navigate to: http://192.168.1.69:1880

9.4 Install Dashboard 2.0

5. In Node-RED, click the hamburger menu (☰) in the top-right corner.
6. Select "Manage palette".
7. Click the "Install" tab.
8. Search for: @flowfuse/node-red-dashboard
9. Click "install" next to the first result, then confirm.

13. Quick Start Guide: Daily Operations

13.1 Starting the Gateway (From Power Off)

10. Ensure the nRF52840 dongle is plugged into a USB port on the Pi.
11. Ensure the Ethernet cable is connected.
12. Connect the USB-C power supply.
13. Wait 60-90 seconds for boot to complete.
14. All services start automatically - no manual intervention needed.

13.2 Accessing the Interfaces

Interface	URL
OTBR Web GUI	http://192.168.1.69
Node-RED Editor	http://192.168.1.69:1880
Dashboard	http://192.168.1.69:1880/dashboard
SSH Access	ssh pi@192.168.1.69 (password: raspberry)

13.3 Checking System Status

SSH into the Pi and run:

```
sudo systemctl status otbr-agent    # Thread Border Router
sudo systemctl status otbr-web      # OTBR Web GUI
sudo systemctl status mosquitto     # MQTT Broker
sudo systemctl status nodered       # Node-RED
sudo ot-ctl state                   # Thread network state (should be
'leader')
```

13.4 Shutting Down Safely

⚠ **WARNING:** Never just unplug the power! This can corrupt the SD card.

SSH into the Pi and run:

```
sudo shutdown -h now
```

Wait until the green ACT LED stops blinking (about 10 seconds), then disconnect power.

13.5 Restarting Services

If a service is not responding, restart it:

```
sudo systemctl restart otbr-agent
sudo systemctl restart otbr-web
sudo systemctl restart mosquitto
sudo systemctl restart nodered
```

12. Final System Configuration Summary

12.1 Thread Network Credentials (SAVE THESE!)

These credentials must be programmed into your QPG6200L device firmware for them to join the Thread network:

Parameter	Value
Channel	18
PAN ID	0x1905
Network Key	67b33c6dca3b4ccf96a56c318b887c39

⚠ **WARNING:** Your network key is like a password. Keep it secure. Anyone with this key can join your Thread network.

12.2 System Credentials

System	Username	Password
Raspberry Pi SSH	pi	raspberry

ℹ **NOTE:** Change the default password for production use: ssh in and run 'passwd'

— End of Document —

Document created: March 9, 2026